Parallel Architectures and Parallel Algorithms
CE502
Summer 2025/2026 **Project: Part-B**

Project – Part B: Sequential and Parallel Image Negative Using MPI (10 Points)

Develop a program that computes the negative of a grayscale PGM image using both sequential and MPI implementations. The program must measure and compare their execution times.

Run the program as follows:

```
mpiexec -n 4 ./image_negative input.pgm negative.pgm
```

The input and output image filenames are provided to the program as `argv[1]` and `argv[2]`. The program should validate the arguments and display a usage message when necessary.

Requirements

- Rank 0 reads the input image and creates the required output images using the provided PGM functions.
- Rank 0 first computes the negative of the complete image sequentially and measures:

$$T_{seq}$$

- The image negative is computed using:

$$I_{\text{negative}} = 255 - I_{\text{input}}$$

- For the MPI version, Rank 0 broadcasts the image width and height to all processes.
- Divide the image into equal chunks of complete rows. You may assume:

$$h \bmod \text{size} = 0$$

- Distribute the row chunks among the processes.
- Each process computes the negative of the pixels in its assigned chunk.
- Collect the processed chunks on Rank 0 and write the resulting negative image.
- Measure the total communication time for the broadcast, scatter, and gather operations:

$$T_{comm} = T_{broadcast} + T_{scatter} + T_{gather}$$

Parallel Architectures and Parallel Algorithms
CE502
Summer 2025/2026 **Project: Part-B**

- Measure the parallel computation time T_{comp}, covering only the local image-negative computation.
- Calculate:

$$T_{par} = T_{comm} + T_{comp}$$

$$\text{Speedup} = \frac{T_{seq}}{T_{par}}$$

- Use the time of the slowest process when determining the MPI communication and computation times.
- Exclude image reading, image creation, and file writing from the measured execution times.
- Rank 0 must print T_{seq}, T_{comm}, T_{comp}, T_{par}, and the speedup.
- Release all dynamically allocated memory and terminate MPI correctly.

Useful APIs

```c
MPI_Init()
MPI_Comm_rank()
MPI_Comm_size()
MPI_Wtime()
MPI_Bcast()
MPI_Scatter()
MPI_Gather()
MPI_Reduce()
MPI_Finalize()
```

Relevant PGM functions:

```c
pgm_read()
pgm_create()
pgm_write()
```

Use `MPI_MAX` with an appropriate collective operation to determine the maximum measured time across all processes.